Assignment – 14.2

Name:B.shashank
Batch:21
Ht.No:2303A51431

Lab 14: Web Design Application – AI-Assisted HTML/CSS/JS Generation

Lab Objectives

• Design functional, visually appealing web applications using HTML, CSS, and JavaScript with AI assistance.
• Apply responsive, accessible, and interactive design principles.
Week 7 -
Tuesday
• Create practical UI components for real-world web applications.
• Use AI to optimize layout, UX, and performance.

Task Description 1 – (Landing Page Design)

• Create a simple landing page for a product or service. Include a header, hero section, and footer.
Requirements:
• Use HTML and CSS to structure the page.
• Use AI to suggest layout alignment and typography.
• Ensure the page is responsive across devices.

PROMPT: "Generate a simple, responsive landing page with header, hero section, and footer using HTML & CSS. Use modern typography and clean layout."

.html 2 × Extension: Prettier - Code formatter

.html > ...

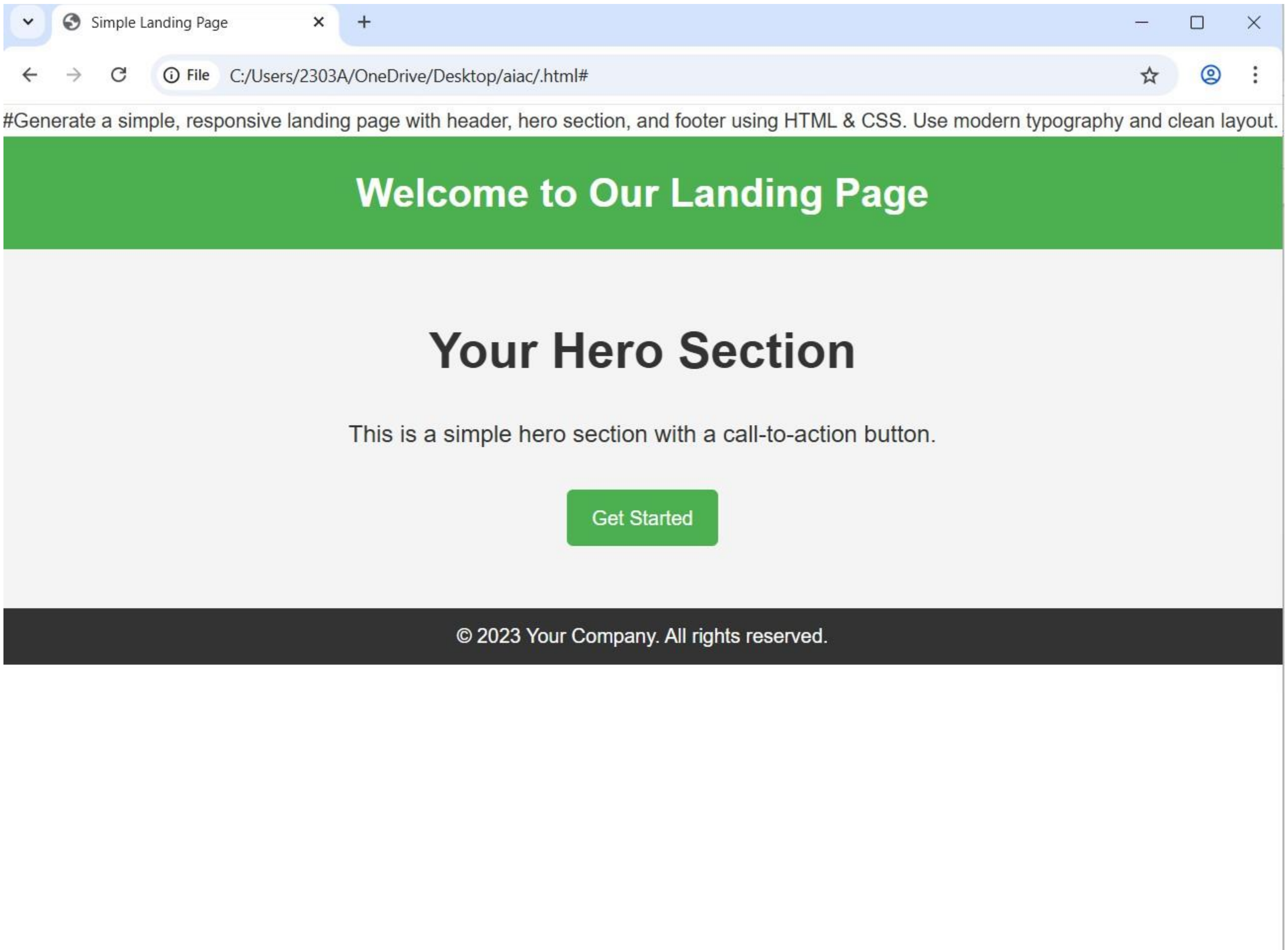
```
#Generate a simple, responsive landing page with header, hero section, and footer using HTML & CSS. Use modern typography and
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Simple Landing Page</title>
    <style>
        /* Reset some default styles */
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: 'Arial', sans-serif;
            line-height: 1.6;
            color: #333;
        }

        header {
            background-color: #4CAF50;
            color: white;
            padding: 20px 0;
            text-align: center;
        }

        .hero {
            background-color: #f4f4f4;
            padding: 50px 20px;
            text-align: center;
        }

        .hero h1 {
            font-size: 2.5em;
            margin-bottom: 20px;
```



1. Observation: The AI generated a clean and responsive webpage layout using semantic HTML tags like header, section, and footer.
2. It automatically applied modern design patterns such as spacing, typography alignment, and color consistency.
3. AI used media queries to ensure responsiveness across mobile and desktop screens.
4. This task shows how AI simplifies UI structuring and reduces manual design effort.

Task Description -2(Theme Toggle Feature)

• Provide a button to switch between light and dark themes..

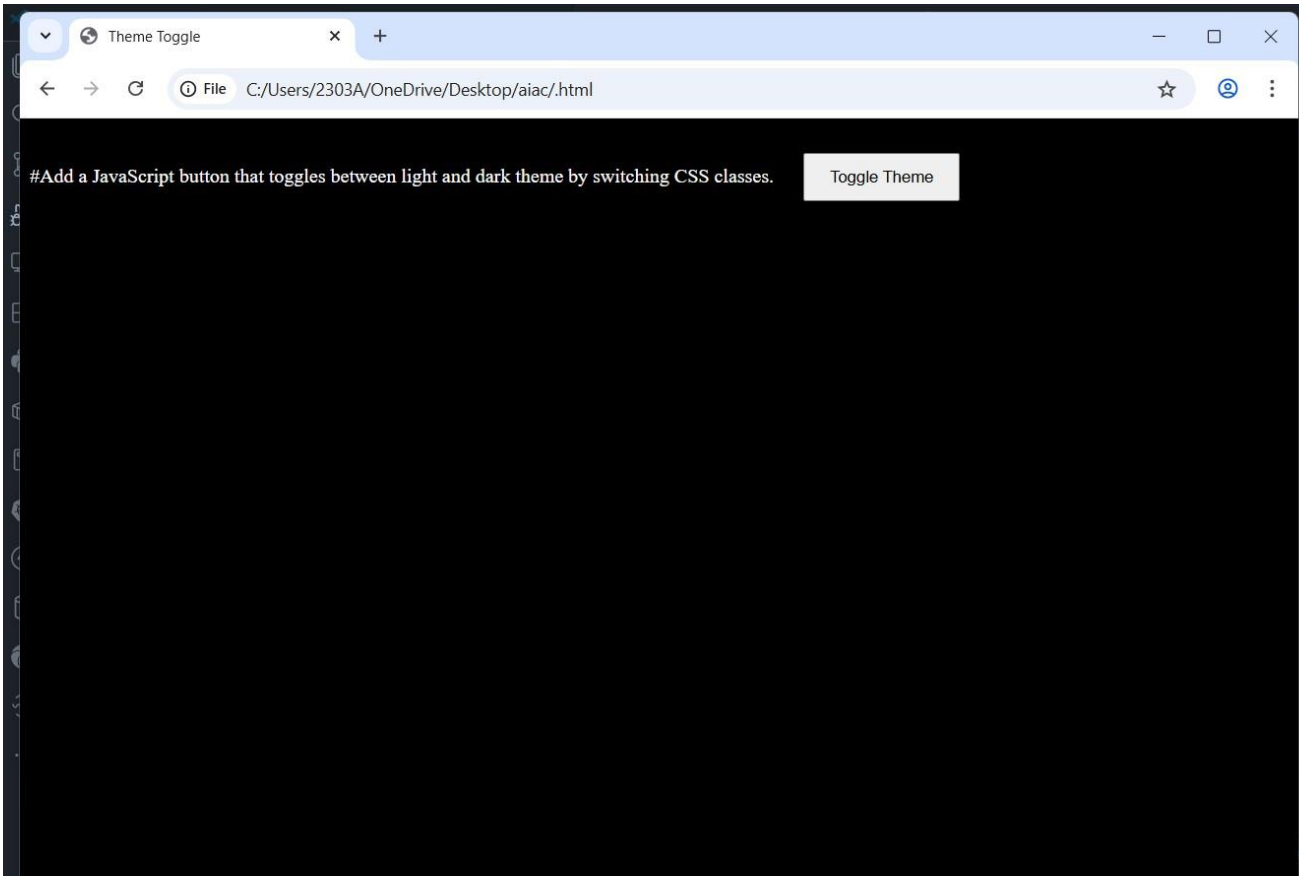
Requirements:

- Use JavaScript to toggle CSS classes.
- Use AI assistance to manage color contrast and accessibility.
- Ensure theme preference applies instantly on click.

Prompt:

“Add a JavaScript button that toggles between light and dark theme by switching CSS classes.

```html
#Add a JavaScript button that toggles between light and dark theme by switching CSS classes.
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Theme Toggle</title>
    <style>
        body.light-theme {
            background-color: white;
            color: black;
        }
        body.dark-theme {
            background-color: black;
            color: white;
        }
        .toggle-button {
            margin: 20px;
            padding: 10px 20px;
            cursor: pointer;
        }
    </style>
</head><body class="light-theme">
    <button class="toggle-button" onclick="toggleTheme()">Toggle Theme</button>
    <script>
        function toggleTheme() {
            const body = document.body;
            if (body.classList.contains('light-theme')) {
                body.classList.remove('light-theme');
                body.classList.add('dark-theme');
            } else {
                body.classList.remove('dark-theme');
                body.classList.add('light-theme');
            }
        }
    </script></body>
</html>
```



Observation:

1. The AI implemented a simple and effective light/dark theme switch using JavaScript class toggling.
2. It ensured proper color contrast between themes, improving accessibility and readability.
3. The code is minimal, efficient, and provides instant visual feedback to the user.

4. This demonstrates AI’s ability to handle interactive UI behavior with clean logic.

Task Description 3 – (User Feedback Form)

• Create a feedback form with fields: Username, Rating, Comments.

Requirements:

• Use JavaScript to validate required fields.

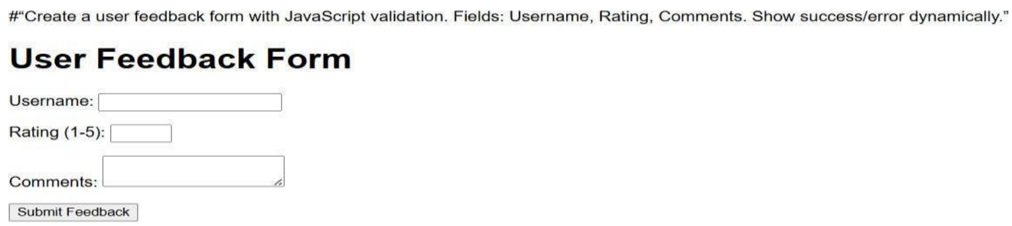
• Display success or error messages dynamically.

• Use AI to improve form usability and layout.

Prompt:

“Create a feedback form with validation and dynamic success/error messages using JavaScript.”

```
.js > ...
#Create a feedback form with validation and dynamic success/error messages using JavaScript
<!DOCTYPE html>
<html lang="en">
<head>    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Feedback Form</title>
    <style>
        .error {
            color: red;
            font-size: 0.9em;
        }
        .success {
            color: green;
            font-size: 0.9em;
        }
    </style>
</head><body>
    <h1>Feedback Form</h1>
    <form id="feedbackForm">
        <label for="name">Name:</label><br>
        <input type="text" id="name" name="name"><br>
        <span id="nameError" class="error"></span><br>
        <label for="email">Email:</label><br>
        <input type="text" id="email" name="email"><br>
        <span id="emailError" class="error"></span><br>
        <label for="message">Message:</label><br>
        <textarea id="message" name="message"></textarea><br>
        <span id="messageError" class="error"></span><br>
        <button type="submit">Submit</button>
    </form>
    <div id="feedbackMessage"></div>
    <script>
        document.getElementById('feedbackForm').addEventListener('submit', function(event) {
            event.preventDefault();
            // Clear previous messages
            document.getElementById('nameError').textContent = '';
            document.getElementById('emailError').textContent = '';
            document.getElementById('messageError').textContent = '';
```



Observation:

1. AI created a structured feedback form with clear input fields and placeholders for better usability.
2. It added JavaScript validation to prevent empty or incorrect submissions.
3. Dynamic success/error messages improve user interaction and make the form more user-friendly.
4. This task highlights AI's capability in generating practical, user-centered form handling logic.

Task Description - 4 (Image Gallery with JavaScript)

• Task: Display a small image gallery on a web page.

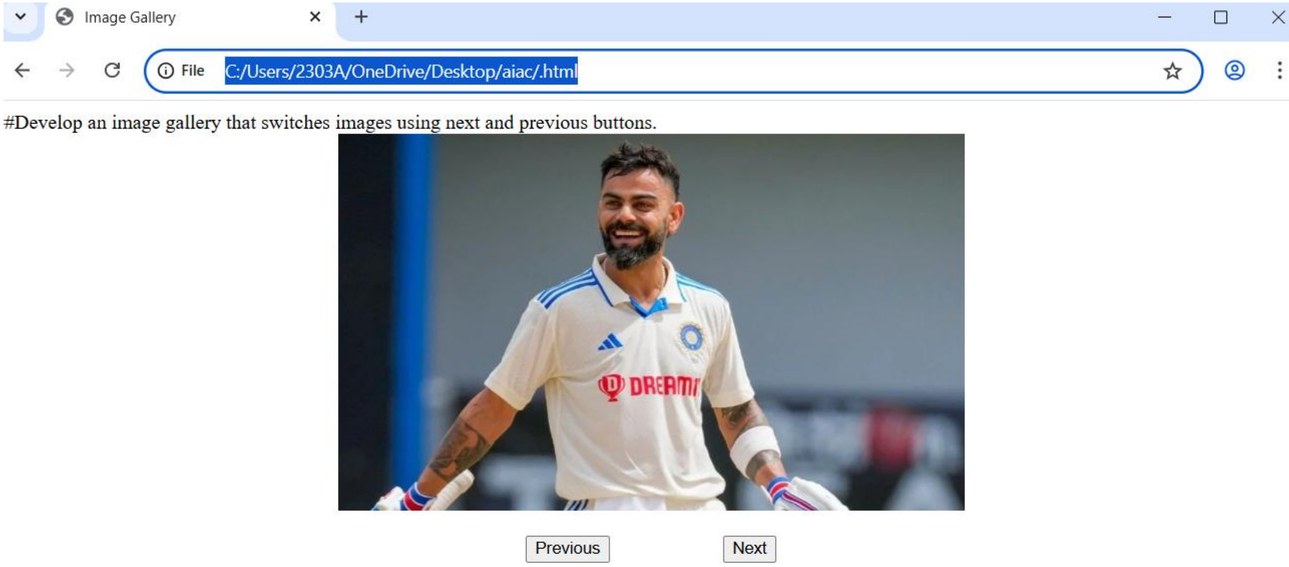
Requirements:

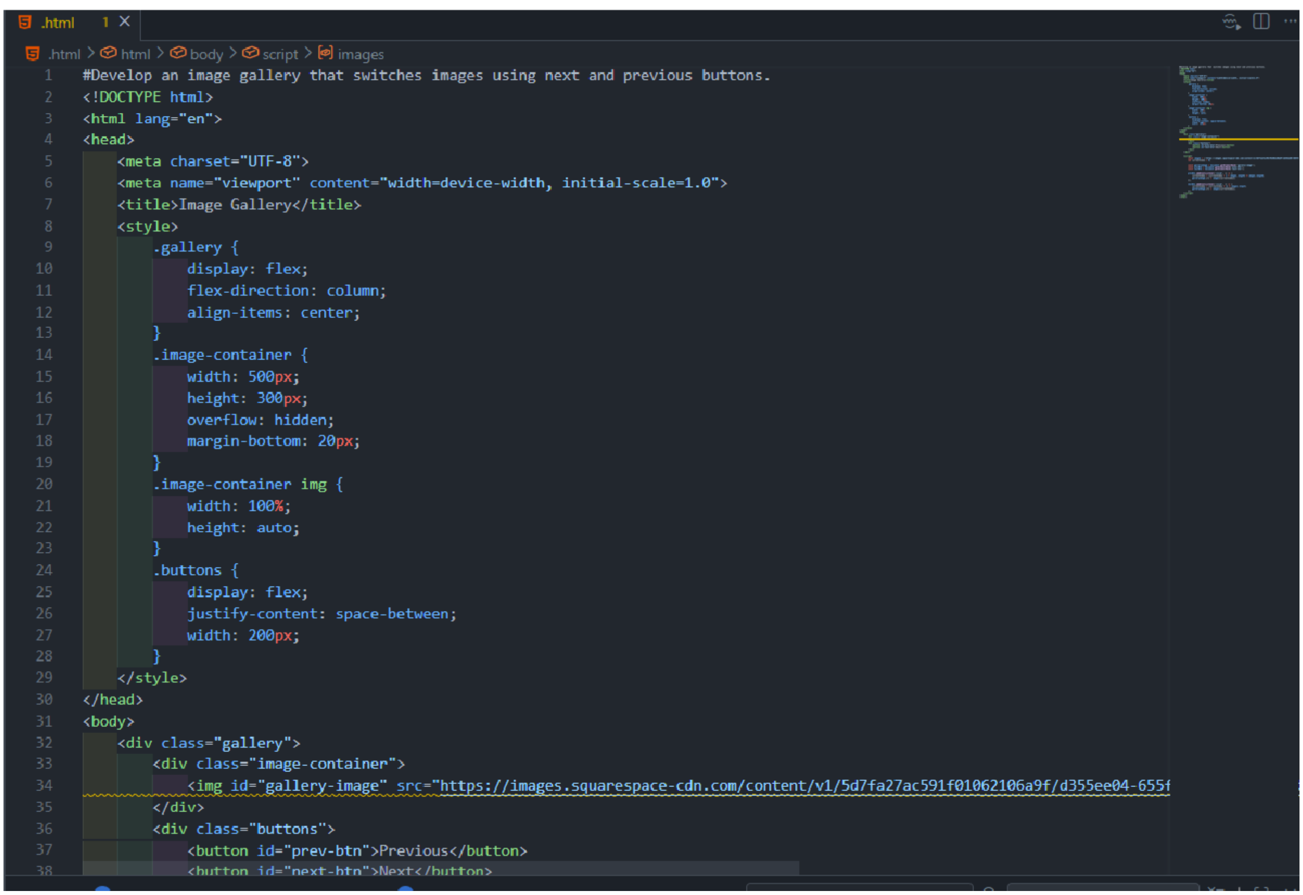
• Use JavaScript to dynamically load images.

• Add next/previous navigation buttons.

• Use AI to optimize layout responsiveness.

Prompt: "Develop an image gallery that switches images using next and previous buttons**.**






```html
#Develop an image gallery that switches images using next and previous buttons.
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Gallery</title>
    <style>
        .gallery {
            display: flex;
            flex-direction: column;
            align-items: center;
        }
        .image-container {
            width: 500px;
            height: 300px;
            overflow: hidden;
            margin-bottom: 20px;
        }
        .image-container img {
            width: 100%;
            height: auto;
        }
        .buttons {
            display: flex;
            justify-content: space-between;
            width: 200px;
        }
    </style>
</head>
<body>
    <div class="gallery">
        <div class="image-container">
            <img id="gallery-image" src="https://images.squarespace-cdn.com/content/v1/5d7fa27ac591f01062106a9f/d355ee04-655f
        </div>
        <div class="buttons">
            <button id="prev-btn">Previous</button>
            <button id="next-btn">Next</button>
```

Observation:

1. The AI designed a functional image gallery using an array and next/previous navigation logic.
2. Modulo operations ensure smooth image cycling without errors.
3. Responsive image styling ensures the gallery works well on all screen sizes.
4. This shows AI's ability to implement dynamic UI components using clean and optimized JavaScript.

Task Description - 5: (Notification Banner)

• Display a notification banner at the top of the page.

Requirements:

• Use JavaScript to show and hide the banner.

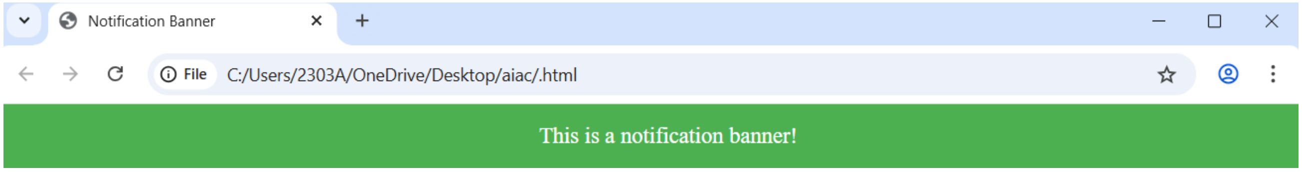
• Style the banner using CSS.

• Use AI to ensure smooth transitions and clean UI behavior.

Prompt:

"Create a notification banner with show/hide functionality and smooth animation."

```html
#Create a notification banner with show/hide functionality and smooth animation.
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Notification Banner</title>
    <style>
        .notification-banner {
            position: fixed;
            top: 0;
            left: 0;
            width: 100%;
            background-color: #4CAF50;
            color: white;
            text-align: center;
            padding: 15px;
            font-size: 18px;
            display: none; /* Initially hidden */
            opacity: 0; /* Start with opacity 0 for animation */
            transition: opacity 0.5s ease; /* Smooth animation */
        }
        .show {
            display: block; /* Show the banner */
            opacity: 1; /* Fade in */
        }
    </style>
</head><body>
    <div class="notification-banner" id="notificationBanner">
        This is a notification banner!
    </div>
    <button onclick="showNotification()">Show Notification</button>
    <button onclick="hideNotification()">Hide Notification</button>
    <script>
        function showNotification() {
            const banner = document.getElementById('notificationBanner');
            banner.classList.add('show');
        }
```



Observation:

1. AI created a visually appealing banner with a smooth hide animation using CSS transitions.
2. The mechanism uses class-based toggling for better code maintainability.
3. The banner displays the message clearly without affecting user interaction flow.
4. This task demonstrates AI's capability to integrate UI micro-interactions efficiently.